



7.2 Student Guide: PowerShell Scripting

Class Overview

Today's class will introduce you to the PowerShell command-line scripting language. You will learn how to use PowerShell to create and execute various scripts for system administration.

Class Objectives

By the end of today's class, you will be able to:

- Use basic PowerShell cmdlets to navigate Windows and manage directories and files.
- Use PowerShell pipelines to retrieve Windows system event logs.
- Combine various shell-scripting concepts such as cmdlets, parameters, conditions, and importing files with defined data structures.

Lab Environment

Lab Details

In this module, you will use the Windows lab environment. RDP into the **Windows RDP host machine** using the following credentials:

- Username: azadmin
- Password: p@ssw0rdp@sww0rd

Open up the Hyper-V Manager in the Windows RDP host machine to access the nested virtual machines:

Windows 10 Machine

- Username: sysadmin
- Password: cybersecurity

Windows Server Machine

- Username: sysadmin

- Password: p4ssw0rd*

For today, the instructor lectures, demonstrations, student activities, and reviews will all be completed using the **Windows RDP host machine**.

We won't need the nested Hyper-V virtual machines until the third day of this module.

Slideshow

The slides for today can be viewed on Google Drive here: [7.2 Slides](#)

- Note: Editing access is not available for this document. If you or your students wish to modify the slides, please create a copy by navigating to File > "Make a copy..."
-

01. Welcome and Overview

In the previous class, we covered:

- Auditing processes with Task Manager
- Using CMD to create files
- Creating a report with `wmic` in the command line
- Auditing unwanted startup applications and services
- Enumerating local users, groups, and current local password policies
- Creating new regular and administrative users and setting the local password policy
- Scheduling tasks using Task Scheduler

We used CMD to execute a lot of these tasks. Why?

- The commands are short and simple. System administration tasks are easily repeatable when the commands are shorter.

- Given its simple commands, CMD is much easier to learn than other command lines, like PowerShell, which is a more complex, but also more powerful, scripting language.

While CMD has its benefits, it wasn't designed to allow system administrators to execute complex operations and procedures. PowerShell was designed as a powerful language for executing, automating, and customizing the most demanding and difficult tasks.

What is PowerShell?

Today, we will cover PowerShell, the successor to CMD.

- PowerShell is a powerful scripting language that allows people to locally and remotely manage Microsoft products, which come with built-in PowerShell support.
- Because the Microsoft enterprise suite of products is the most widely used by organizations, it is critical that system administrators and security professionals know how to use PowerShell.
- PowerShell can be used to lock down and harden enterprise networks. Since PowerShell's capabilities are powerful enough to manage enterprise Windows systems, they can be leveraged by offensive security professionals and malicious actors.

PowerShell plays an important role in a cyber professional's work:

- **PowerShell in system administration:** PowerShell can be used to manage everything in a Microsoft enterprise environment, including:
 - Windows Server: Microsoft's central server for domain and networking services.
 - This includes managing users and network resources, such as a file-sharing server.
 - Certain types of Windows servers, such as domain controllers, are central components to an enterprise network. They maintain control and permissions of all users within Microsoft-based enterprises. It's vital to keep these servers secure. We'll learn more about domain controllers in the next class.
 - A compromised domain controller is a worst-case scenario for most organizations.
 - Windows 10: Microsoft's personal and professional computer operating system. Windows 10 is the most commonly used operating system by individuals and

within large enterprise environments. It comes in various editions, such as Pro or Enterprise, each adding some functionality, such as access to Hyper-V.

- Note that because Windows is the most widely used operating system, it is also the one attackers most often try to find vulnerabilities for.
- Windows 7 is officially unsupported by Microsoft. This means that if a vulnerability is found for Windows 7, Microsoft may not release a patch for those systems to automatically retrieve and update.
- Office365: Microsoft's line of subscription office services.
 - You've probably used one of these products, such as Microsoft Word or Excel.
- Azure: Microsoft's cloud services.
 - Azure provides a wide range of cloud services such as virtual private clouds or cloud-based virtual machines, like the one you're using right now.

PowerShell in cybersecurity: For any organization using Windows-based environments, it's important for cybersecurity professionals to know some PowerShell.

- Defensive security: PowerShell can be used to manage and audit logs. There are many commands for interacting with Windows Event logs. We will look at these later.
 - PowerShell can also be used to harden the security on Windows hosts and servers. There are many modules and scripts that extend PowerShell's powerful functionality to enforce cybersecurity policy.
- Offensive security: PowerShell is often used as a "living off the land" tool, meaning a tool that exists on the target's computer that can be leveraged by attackers.
 - Once a system is breached, PowerShell is often used to retrieve a wide range of information within a network, such as user and server names. It can also be used to access and maintain access on other networked machines, if they are not properly secured.

Comparing PowerShell and CMD

:warning: **Reminder:** This entire day is done with the **Windows RDP host machine**.

While CMD allows the user to interact with Windows, its functionality is limited.

- CMD's output is only available in simple text format. Our only option for unsupported file formats is to edit the output with meticulous character replacing. It's similar to using

Write-Over to edit a printed document over and over.

- CMD's command flags can be ambiguous and confusing. Each tool has its own flags. Let's look at two commands that use the `/s` and `/d` flags:
 - `shutdown /s` shuts down a computer.
 - `freekdisk /s` specifies the name of an IP or remote computer to check disk space.
 - `shutdown /d` specifies a reason for shutdown.
 - `freedisk /d` specifies which disk drive to check.

PowerShell remedies this inconsistency with clearly defined parameters for each command. This makes PowerShell more predictable, easier to write, and easier to read. Below are some examples:

- `Stop-Computer -Confirm`
 - This command will ask you to confirm shutting down the computer. However, running tasks may prevent the shutdown from occurring.
- `Stop-Computer -Force`
 - This command will immediately shut down the computer regardless of what is running. This will ignore the usual tasks that can prevent a graceful shutdown, such as large file transfers and 3D applications.

PowerShell Piping for CMD's Unsupported Operations

Another issue with CMD is that it doesn't easily support certain operations.

In the next example, we'll run a complex CMD batch file and compare it to a simple PowerShell alternative.

If we want to find the file sizes of subdirectories in `C:\Windows\System`, we need a complex CMD batch file.

- Open CMD and put the two windows next to each other.
- In CMD, run `type C:\windows\filesizes.bat` to show its contents.

- Your output should be:

```
Unset
@echo off
set size=0
for /r %%x in (System\*) do set /a size+=%%~zx
echo %size% Bytes
```

- The batch file would need to be in the `C:\Windows` directory when the command is run.
- In CMD, run `C:\Windows\filesizes.bat` and note the results (your results may vary).
 - **Note:** This may take a while.

```
Unset
C:\Windows>filesizes.bat
30795 Bytes
```

PowerShell can do this same task using a simple pipe (`|`). Piping sends the output of one command as the input for another.

- In PowerShell, type `dir C:\Windows\System -Recurse`.
 - We can run `dir C:\Windows\System -Recurse` to grab the contents of all current directory and subdirectories, much like `ls -R` grabs the current directory and subdirectory contents in Linux.
 - Then, we can pipe that output to a command that will measure the files for us.
- Type the rest of the command: `dir C:\Windows\System -Recurse | Measure-Object -Sum Length`
 - The `Measure-Object` command receives all the previous inputs and returns the sum of all the file sizes. It does this more accurately than CMD.
- Run the command to see the results. Optionally, view the contents of the subdirectory in `System` (there should be a subdirectory, `Speech`, with files whose sizes add up to

what's shown below).

Your results should be similar to:

```
Unset
Count      : 3
Average    :
Sum        : 31039
Maximum    :
Minimum    :
Property   : length
```

Differences Between CMD and PowerShell

- The CMD batch script required the default file size be set to 0.
- It took a long time to find the individual file sizes within the subdirectory.
- It then read each of those text strings and added them to the original file size.

This script is inefficient and challenging for others to read.

The PowerShell pipe knew that file sizes were a type of property that a directory or file can have.

- `dir` grabs all of the directories and files recursively.
- Then, the `Measure-Object` is used to only retrieve the file sizes of the files and directories.
- Then, we simply used an argument, `-Sum Length`, to total the file sizes.

In PowerShell, things that have defined properties, such as files having file sizes, are known as **objects**.

PowerShell and Objects Demo

Note that objects are things with properties or attributes.

- PowerShell innately understands and interacts with everything as an object with attributes or properties.

Let's look at the following example:

- Run `ls C:\Windows`.
- Each file and directory is being processed by PowerShell as an object, each with its own properties, such as file size or file name.
 - For example: The file's name is a property of the file. We can define it as: `file.name`.

Now we'll use a pipe to retrieve only the objects whose `name` property has the word "system" in it.

- Type: `ls C:\Windows | Where-Object {$_.name -like "*system*"}`. (Tab completion can be faster than typing the command.)
 - In this example, PowerShell will return the contents of a directory as objects.
 - It will look through their `.name` properties for the word "system."
 - `$_` means previous object. In this case, the previous objects are all of the files and directories under `C:\Windows`.
- In summary, we are passing the objects of `ls C:\Windows` to the `Where-Object` command to be filtered down to names that contain "system."

Run the command to show the results:

Unset

```
PS C:\Users\azadmin> ls C:\Windows | Where-Object {$_.name -like
"*system*"}
```

Directory: C:\Windows

Mode	LastWriteTime	Length	Name
----	-----	-----	----

```
d-----      3/19/2019      4:52 AM              System
d-----      1/30/2020      5:58 PM              System32
d-----      3/19/2019      6:22 AM              SystemApps
d-----      1/9/2020       9:22 PM              SystemResources
-a-----      3/19/2019      4:49 AM          219 system.ini
```

- **Note:** Results may vary slightly.

Understanding how objects work is important for system administrators and security professionals interacting with Microsoft products.

More PowerShell Advantages and Aliases

Let's cover some more of PowerShell's advantages:

- PowerShell commands are extensively documented. There are multiple ways to make sure you're using the right commands with PowerShell's extensive internal documentation system.
- PowerShell uses **aliases** to mimic Unix commands like `ls` and `cat`.
 - Running `Get-Alias ls` will provide the PowerShell cmdlet counterpart for Unix commands.
 - **Note:** These commands function differently than their Unix counterparts when it comes to options and arguments. For example, `ls` works but `ls -la` does not.
- Despite being a Microsoft product, PowerShell is officially open source and available on GitHub. In contrast, there is no source code available for `cmd.exe` and writing tools for CMD are limited compared to PowerShell.

02. Intro to PowerShell

We are going to continue our system administration tasks by readying a Windows workstation for new users.

Remember the scenario we used in the previous lesson:

- Our CIO tasked us with readying the Windows workstation through the command line and GUI. Today, the CIO asked us to use PowerShell to complete the following activities:
 - Navigate Windows with the PowerShell command line.
 - Create logs with PowerShell.
 - Eventually, these logs can be transferred to a central logging repository for SIEM integration. We'll learn about SIEM more in a later module.
 - Create scripts with PowerShell.

PowerShell Syntax: Cmdlets

Let's get started by looking at PowerShell's syntax.

- First, we are going to learn how to use PowerShell commands, otherwise known as cmdlets (pronounced "command-lets").

Cmdlets are the combinations of PowerShell verbs and nouns. This is part of what makes PowerShell easy to read and use.

- Refer to the CheatSheet file for this week's lessons: [7.2 CheatSheet](#)
- Open the file and look at the Common PowerShell Commands table while following this demo.

Cmdlet	Function	Equivalent Command
Set-Location	Changes to specified directory	cd
Get-ChildItem	Returns current directory's contents	ls, dir
New-Item	Makes a new file or directory	touch, mkdir

Remove-Item	Deletes a file or directory	rm, rmdir
Get-Location	Retrieves path to current directory	pwd
Get-Content	Returns file contents	cat, type
Copy-Item	Copies a file from one given location to another	cp
Move-Item	Moves a file from one given location to another	mv
Write-Output	Prints output	echo
Get-Alias	Shows aliases for the current session	alias
Get-Help	Retrieves information about PowerShell commands	man
Get-Process	Gets processes running on local machine	ps
Stop-Process	Stops one or more defined processes	kill
Get-Service	Gets a list of services	service --status-all

We'll start with some navigation commands and then work our way to creating Windows system event logs.

PowerShell Verb-Nouns Demo

Consider the following scenario:

User **Alex** left our company. We need to remove their user account from the system. However, Alex has an unfinished **reports** directory on their account.

Before removing the account, we need to move the **reports** directory and files from their desktop directory to a directory that will remain after we remove the account.

1. Let's change the current working directory to the **reports** directory:
 - Run **Set-Location C:\Users\Alex\Desktop**.
 - **Tip:** Use tab completion to enter commands quickly.
 - These commands follow a verb-noun format.
 - The verb in this case is **Set**.
 - The noun is **Location**.
 - We are setting the location of our terminal to **C:\Users\Alex\Desktop**.
 - This is the same as **cd** in Linux, an option we can also use with PowerShell.
2. We want to move the **reports** logs from **Alex's** desktop to the **C:** drive. We can do this with **Move-Item**.
 - Run **Move-Item reports C:**.
 - The verb, **Move**, will act on a noun, **Item**, which can be a file or directory.
 - This is the same as **mv** in Linux.
3. We can check the contents by using the cmdlet **Get-ChildItem**.
 - Run **Get-ChildItem C:** to display that **reports** now shows up under **C:**.

- This command will retrieve all the child items of the designated parent path, `C:\`.
 - The verb is `Get` and the noun is `ChildItem`.
 - You can also use `ls`, just like in Linux.
4. We can create an **item** by running `New-Item demo_file`
- By default, this cmdlet creates a file as opposed to a directory. This default behavior is the same command as `touch` in Linux. We'll see in a few moments how it can be used to create directories.
 - Run the command and then run `Get-ChildItem` to show the directory's contents.
5. Now that we've moved our `reports` directory outside of the user's desktop directory to `C:\`, we can delete the user account but retain the reports.
- Run `Remove-Item demo_file` to delete the file.
 - This cmdlet functions similarly to `rm` in Linux.
 - Run `Get-ChildItem` to show it's no longer there.

Now we can navigate anywhere, and create and delete files. Let's look at some slightly more complex commands.

Parameters

In the previous demo, we used the `New-Item` cmdlet to create an object with the item-type of file. This is the default behavior of `New-Item`—to act like the Linux command `touch`.

If we want the item-type to be a directory, with a specific name and location (like the `mkdir` command), we can add a parameter `New-Item`.

Let's use `New-Item` with parameters to set up a new directory for storing the logs we generate.

- Run `New-Item -Path "C:\" -Name "Logs" -ItemType "Directory"`.

- This command is a little more complicated but will make sense once you look closer. This is the beauty of PowerShell's predictable and descriptive commands.
 - `New-Item` is similar to `Move-Item` in that it can create a file or, now, a directory.
 - `-Path` is the parameter specifying which directory we want our new item created in. In this case, `C:\`.
 - `-Name` is the parameter specifying what we want to name this new item. In this case, `Logs`.
 - `-ItemType` is the type of item we want to create. If we don't specify `"Directory"`, it will default to file, like before.
- The parameters `-Name` and `-ItemType` modify the original function of `New-Item` by altering the object it will create.

Now we should see a directory, `Logs`, created in `C:\`.

- Run the command to show the confirmation output.
- We can verify this once again by using the cmdlet `Get-ChildItem`.
- Run `Get-ChildItem C:\` to display the newly created `Logs` directory.

With the power of PowerShell parameters, we can achieve the effects of `touch` and `mkdir` commands with the `New-Item` cmdlet.

Parameter Examples Demo

We'll run through some more examples of parameters in PowerShell.

1. Press the Up key or again type `New-Item -Path "C:\" -Name "Logs" -ItemType "Directory"`.
 - Run the command to show the error message: `New-Item : An item with the specified name C:\Logs already exists.`
2. We can use the parameter `-Force` to run this `New-Item` command without error.

- Run `New-Item -Path "C:\" -Name "Logs" -ItemType "Directory" -Force`.
3. We can use `Get-Command` to search for commands using keywords.
- For example, if we are looking for commands with the word "process" in them, we can use the following:
 - Run `Get-Command -Name *Process`.
 - `*Process` is a wildcard parameter value that we're giving to the parameter `-Name`. This will give us all the commands with the word "process" in them.
 - The output shows all commands with "process" in their name.

```
Unset
  CommandType      Name
Version      Source
-----
Function      Get-AppvVirtualProcess
1.0.0.0      AppvClient
Function      Start-AppvVirtualProcess
1.0.0.0      AppvClient
Cmdlet        Debug-Process
3.1.0.0      Microsoft.PowerShell.Management
Cmdlet        Enter-PSHostProcess
3.0.0.0      Microsoft.PowerShell.Core
Cmdlet        Exit-PSHostProcess
3.0.0.0      Microsoft.PowerShell.Core
Cmdlet        Get-Process
3.1.0.0      Microsoft.PowerShell.Management
Cmdlet        Start-Process
3.1.0.0      Microsoft.PowerShell.Management
Cmdlet        Stop-Process
3.1.0.0      Microsoft.PowerShell.Management
Cmdlet        Wait-Process
3.1.0.0      Microsoft.PowerShell.Management
```

4.

From this list, we see `Start-Process`. Let's run an administrative CMD session from PowerShell.

- Run `Start-Process -FilePath CMD -Verb RunAs`.

5. We can also close the CMD process.

- Run `Stop-Process -Name CMD`.

03. Activity: Move and Create Directories

 M7.2 Activity File: Move and Create Directories

04. Activity Review: Move and Create Directories

05. Generating Windows Event Log Files with Parameters and Pipelines

Reminder: Continue using the **Windows RDP host machine** for this section's demos.

In the previous activity, we used parameters to modify various PowerShell cmdlets. Now we're going to build on parameters further by chaining commands using pipelines.

- Chaining commands is important in system administration and security roles. For example, quickly retrieving and sorting through various logs is a common task.

In the next demo, we will continue our role as a junior system administrator. Our CIO has asked us to retrieve multiple types of logs from our Windows workstation and save them as JSON files in our newly created `C:\Logs` directory, so they can later be imported to a Splunk SIEM for the company's analysts.

For our next demo, we are going to chain cmdlets together to create a PowerShell pipeline that does the following:

- Retrieves Windows logs from Windows Events.
- Transforms the logs to the structured data format JSON.

- Outputs these files to the `C:\Logs` directory we made in the last activity.

Powershell Parameters Demo

1. Let's learn the command for retrieving the Windows Event log from Windows.
 - Run `Get-WinEvent`.
 - The output should show the huge amount of scrolling, invalid logs. This output could take days to finish (or error out).
 - Windows Event logs are recorded instances and information of events that occur in Windows, such as date and time of event, criticality of event, what happened during the event (e.g., a service started), etc.
 - Press Ctrl+C to stop the output.
 - `Get-WinEvent` is a cmdlet that requires parameters to retrieve a manageable amount of logs.
 - For example, adding `-LogName` followed by the name of a log, such as `System`, will only show us the system logs. So, let's clear our screen and try again.
 - Run `clear` in console to clear the screen.
2. Now let's list all log categories instead of every log ever created.
 - Run `Get-WinEvent -ListLog *`.
 - The asterisk `*` is a wildcard indicating everything after the `-ListLog` parameter, so it will return every log that we can retrieve with `Get-WinEvent`.
 - As you can see from the output, the command displays a much more concise and approachable output of log categories, rather an enormous list of every single log.

Unset

```
```PowerShell
```

```

LogMode MaximumSizeInBytes RecordCount LogName

Circular 15728640 4513 Windows PowerShell
Circular 1052672 0 Windows Azure
Circular 20971520 12661 System
Circular 20971520 27813 Security
...

```

- **Note:** Results may vary.

- - :books: Refer to the following link if you're curious about LogMode Circular Logging:
    - [Whatis.com: Circular Logging](https://www.whatis.com/circular-logging)
  - 1. If we want to find specific logs, such as system, we can use the `-LogName` parameter to specify which ones.
    - Run `Get-WinEvent -LogName System` to return system logs.
    - This will produce a lot of results. Enter Ctrl+C to stop the output.
  - 2. Since there are thousands of logs and PowerShell will attempt to retrieve all of them, we need to narrow down that list using parameters.
    - Run `Get-WinEvent -LogName System -MaxEvents 10`.
      - By adding the `-MaxEvents` parameter and the value `10`, we should now see only up to 10 logs.

## Piping Logs to JavaScript Object Notation with ConvertTo-JSON Demo

Now that we used parameters to get the logs we need, we will output them into a file that can be used later by log analysis applications. This is where pipelines come in; however, we also need to pick a file format.

- **JSON (JavaScript Object Notation)** structured data format can be easily read and manipulated by today's most common scripting and programming languages.

- Windows logging systems understand JSON, so we will convert our logs to this file format.

In this demo, we will convert logs to the JSON data format using the `|` piping character.

1. Pipe the output of the `Get-WinEvent -LogName System -MaxEvents 10` into another command that converts the logs to JSON.
  - Run `Get-WinEvent -LogName System -MaxEvents 10 | ConvertTo-Json`.
  - Note the slightly less reader-friendly, but highly structured JSON logs:

Unset

```
PS C:\Users\azadmin> Get-WinEvent -LogName System -MaxEvents 10 |
ConvertTo-Json
```

```
{
 "Id": 13,
 "Version": 0,
 "Qualifiers": null,
 "Level": 4,
 "Task": 1003,
 "Opcode": 0,
 "Keywords": -9223372036854775808,
 "RecordId": 2555,
 "ProviderName": "Microsoft-Windows-Hyper-V-Netvsc",
 "ProviderId": "152fbe4b-c7ad-4f68-bada-a4fcc1464f6c",
 "LogName": "System",
 "ProcessId": 3360,
 "ThreadId": 9036,
 "MachineName": "DESKTOP-U3FCUKI",
 "UserId": null,
 "TimeCreated": "\/Date(1595374458918)\/",
 "ActivityId": null,
 "RelatedActivityId": null,
 "ContainerLog": "System",
 "MatchedQueryIds": [
]
}
```

```

],
 "Bookmark": {
 },
 "LevelDisplayName": "Information",
 "OpcodeDisplayName": "Info",
 "TaskDisplayName": null,
 "KeywordsDisplayNames": [
],
 "Properties": [
 "System.Diagnostics.Eventing.Reader.EventProperty",
 "System.Diagnostics.Eventing.Reader.EventProperty"
],
 "Message": "Miniport NIC \u0027Microsoft Hyper-V Network
Adapter #3\u0027 disconnected"
 }

```

- **Note:** Output may vary.
  - The output of `Get-WinEvent` is now in a JSON format.
- 2. Our logs aren't actually in a file yet, so we need a cmdlet that will send this output to a file.
  - PowerShell has a cmdlet called `Out-File` that uses the `-FilePath` parameter to designate the location for the output file. Let's name this file `RecentSystemLogs.json`.
  - Return to the JSON-converted logs, add the `Out-File` cmdlet, and specify a path with `-FilePath`.
    - Run `Get-WinEvent -LogName System -MaxEvents 10 | ConvertTo-Json | Out-File -FilePath C:\Logs\RecentSystemLogs.json`.
    - This command will send our JSON-transformed output to a file called `RecentSystemLogs.json` in the `C:\Logs` directory. Remember, Windows file paths use backslashes instead of forward slashes.

3. Verify that this command worked:

- Navigate to `C:\Logs` with `Set-Location C:\Logs`.
- Run `Get-ChildItem` to confirm `RecentSystemLogs.json` exists in the current directory.
- Run `Get-Content .\RecentSystemLogs.json` to confirm that we now have logs in the `C:\Logs` directory.

---

## 06. Activity: Generating Windows Event Log Files with Parameters and Pipelines

 [M7.2 Activity File: Generating Windows Event Log Files with Parameters and Pipelines](#)

---

## 07. Break

---

## 08. Activity Review: Generating Windows Event Log Files with Parameters and Pipelines

---

## 09. Scripting with PowerShell

Recall that we highlighted the importance and convenience of Linux scripting in our recent system administration modules.

- Scripts allow system administrators and security professionals to automate and execute basic and advanced procedures and operations.

- We can do everything from setting basic firewall rules to standing up entire cloud virtual machine environments with networking, storage, and users.

PowerShell, like Linux, allows us to script many commands in sequence.

- For example, suppose you were asked to set up Windows workstations for users in the accounting department. You could create a script to do the following, in this order:
  1. Pull sensitive accounting data and files to a specified directory from a file server.
  2. Download AppLocker, a program for limiting and controlling access to files for certain users and groups.
  3. Deploy application control policies for AppLocker to restrict user access to the directory so only people in the **accounting** group can access it.

In the following demonstration, we will use scripts to remove unnecessary applications and potential security hazards from Windows workstations.

### **Restaging the Chocolatey Apps**

If you ran the package removal scripts, you can restage your Choco Apps by running the corresponding script for both the activity and demo:

- [stagebloat.ps1](#)

### **Demo Scenario: Scripting the Removal of Unnecessary Packages**

In this demonstration, we will create a script in Git Bash's Nano to remove Microsoft Skype.

Note the following about this Windows RDP host machine:

- This Windows workstation has been created to simulate a previous user having installed a bunch of applications that are considered largely unnecessary and potential vectors of attack.
- These settings include telemetry tracking and advertising IDs, and default installed applications including Skype.
- We want to remove these applications to reduce the attack surface area for this workstation.

- The following article details potential security and privacy vulnerabilities of seemingly harmless and popular applications:
  - [ZDNet: North Korean hackers infiltrate Chile's ATM network after Skype job interview](#)
- Most importantly, it's annoying to see them start up when we boot up the machine!

We will look at the steps needed to remove Skype with PowerShell. These steps will later be used in a PowerShell script to remove many applications at once.

In the upcoming demo, we have been tasked by our CIO with doing the following:

- Create a PowerShell script file and execute it.
- Use a cmdlet to import various items to interact with in a PowerShell script.
- Use a `foreach` loop to go through each item in an imported CSV file.

## Understanding PowerShell Scripts

Understand that PowerShell, by default, does not allow scripts to run. This is a security measure to prevent malware, attackers, and improperly constructed scripts from immediately executing many commands at once, potentially compromising a system.

Our machines have already been configured to allow scripts to run, however.

This means that we can create and run PowerShell scripts for automation. Although this makes it easier for us, it does pose a security risk to be able to freely run scripts.

If we are infiltrated or run a malicious script by accident, we could potentially lose complete access to our machine and all of our data.

Don't worry though, because in the next lesson, we'll look at how to monitor PowerShell scripts. For now, let's look at how to create our first PowerShell script.

## Creating and Executing a PowerShell Script to Remove Applications

PowerShell script files end with a `.ps1` extension.

1. Using Git Bash, move into our previously created `Scripts` directory:
  - `cd C:\Scripts`

- We will be saving our scripts to this directory.
2. Create a new script that we will run in PowerShell:
    - In the Git Bash terminal, run the following command to create a blank file named `example_script.ps1` in the current directory:
      - `nano example_script.ps1`
  3. Now we'll add code to our script to look for the preinstalled app, Skype. We'll set it as a variable here:
    - In the Git Bash Nano document, enter:

```
Unset
$packagename="Skype";
choco info $packagename
```

- Syntax breakdown:
    - `choco`, or "Chocolatey", is a day-to-day Windows package manager similar to the Linux package manager, `apt-get`, that we have seen and used before. System administrators will often use `choco` to install all of the applications their users will need on a Windows workstation.
    - `$packagename="Skype"` declares the variable `$packagename` and assigns "Skype" to it. This will find the `Chocolatey` package, Skype.
4. After we save this file and run it, it should output information about the Choco package, Skype.
    - Save the file using Nano's write out function, so you don't have to close the script.
      - Type `Ctrl+O` and press `Enter` to save the file without closing Nano.
  5. In the PowerShell window, navigate to the same directory (`C:\Scripts`) and run the script:
    - Run `.\example_script.ps1`.

- The console output should look like:

Unset

```
Chocolatey v0.10.15
skype 8.63.0.76 [Approved] Downloads cached for licensed users
Title: Skype | Published: 8/3/2020
Package approved as a trusted package on Aug 03 2020 10:28:35.
Package testing status: Passing on Aug 03 2020 10:05:05.
Number of Downloads: 3050158 | Downloads for this version: 29297
Package url
Chocolatey Package Source:
https://github.com/chocolatey-community/chocolatey-coreteampackag
es/tree/master/automatic/skype
Package Checksum:
'w37/kd0H/A4CrHmqFrEzr/pTYuKhq9KLUThlC0wcd5lJohdKbIDrE7kYWRECJ3ct
7hq3LWME6ZNq3QafDB4gLA==' (SHA512)
Tags: Skype VOIP voice over ip video conferencing admin
Software Site: http://www.skype.com/
Software License:
https://www.microsoft.com/servicesagreement#14e_Skype
Summary: Skype - VOIP
Description: Skype - Install Skype, add your friends as contacts,
then call, video call and instant message with them for free.
Call people who aren't on Skype too, at really low rates.
```

#### ## Notes

The package has the following known issues:

- Skype automatically starts after installation (no known way to disable it yet).
- A desktop shortcut is automatically created.

1 package found.

- 

Point out the **## Notes** section that says:

Unset

The package has the following known issue:  
- Skype automatically starts after installation (no known way to disable it yet).

An automatically starting process, especially one that automatically logs in users and has a history of security issues, should not exist on a remote Windows workstation such as ours.

Now let's look at how we can edit our script from finding a package's information to removing a package from our system.

1. With Nano still open, edit the script to the following:

Unset

```
$packagename="Skype";
choco uninstall -y $packagename
```

- We are now going to use `choco uninstall` to remove Skype.
- The `-y` parameter here will automatically confirm that we want to remove the package, much like `apt-get remove -y <package name>`.

2. Save and run the script.

- Save the script with `CTRL+O`.
- Run the script with `.\example_script.ps1` to show the output:

Unset

```
PS C:\Users\azadmin\Documents> .\example_script.ps1
Chocolatey v0.10.15
Uninstalling the following packages:
Skype
```

```
skype v8.63.0.76
Running auto uninstaller...
Auto uninstaller has successfully uninstalled skype or detected
previous uninstall.
skype has been successfully uninstalled.

Chocolatey uninstalled 1/1 packages.
```

3.

We've successfully removed Skype from our system using a PowerShell script!

- We could remove the Choco packages one by one, but it is more efficient to create a script that will loop through a list of the packages and uninstall them all at once.
- Luckily, we have a file with a list of packages we can remove.
- Navigate to the Choco demo directory with:

- `cd C:\Users\azadmin\Documents\Demo\choco`
- Run `ls` (or `Get-ChildItem`) in PowerShell to show the CSV file, `chocodemo.csv`.

4. Using PowerShell, open and display the [chocodemo.csv](#) file:

- Run `type .\chocodemo.csv`
- Your output should be similar to:

Unset

```
name,description
itunes,"Apple's music application"
vlc,"An application for encoding and playing videos"
[...]
```

- The first line, `name,description`, is the header of the CSV file and contains the types of fields for the other lines.
- The rest of the rows, or records, contain field entries that should match the number and type of header fields.
- For example, "`Apple's music application`" will match up with the `description` header field and `itunes` will match up with the `name` header field.

In the next section, we will take a brief hiatus from our demo to look at CSV files and how we can pair them with other familiar scripting tools to create a more efficient package removal script.

### What's a CSV File?

- **Comma-separated values** (CSV) files are plain text files that contain simply structured data, or fields, separated by commas.
- CSV files often contain structured lines of information made up of rows of data.
- The top line of a CSV file contains the header, the row that describes each field.
- For example, you may have the headers "fruit" and "vegetable," with the corresponding field entries "apple" and "carrot." Another row of entries may have "banana" and "potato."

System administrators and security professionals often use CSV files containing lists of items that they need to parse through.

- For example, a system administrator may use such a file to maintain a list of employee email addresses and usernames.
- A penetration tester might have a list of IP addresses and corresponding domain and subdomain names to use during a test.

Most common programming and scripting languages have ways to import CSV files and **loop** through the items one by one. PowerShell has this capability.

- To loop through this file's items, we have the `foreach` loop.

- The `foreach` loop in PowerShell is similar to the `for` loop you learned about in the Linux module, but it is mainly used for looping through files or read-only structured data.
- Review some cases for which system administrators and security professionals may use a `foreach` loop instead of a `for` loop:
  - A system administrator may have a text or CSV file of usernames they need to loop through to reset each item's password.
  - A penetration tester might have a text file with a list of the most commonly used passwords. They may use a `foreach` loop to try out each password with a known user's username.
- Reiterate that a `foreach` loop is best when iterating or moving through items in some form of a list. Let's look at how we can use it with CSV files.

Let's return to our demonstration and use CSV files and `foreach` loops to optimize our package removal script.

### Using Import-CSV with a foreach Loop Demo

We will be using the `Import-Csv` PowerShell cmdlet to source the CSV file `chocodemo.csv` to run with a demo PowerShell script. If the CSV file has the standard header and record format, `Import-Csv` will prepare the fields to be called by PowerShell.

Let's look at how that works by creating a PowerShell script that outputs each line of the CSV file.

1. First, we'll create a new script. Make sure you're working within the `C:\Users\azadmin\Documents\Demo\choco` directory.
  - In Git Bash, start a new PowerShell script:
    - Run `nano removebloat.ps1`.
2. We are going to use a variable to import our CSV file for our script.
  - On the first line, type `$csv = Import-Csv -Path .\chocodemo.csv`.
    - The variable, `$csv`, gets assigned to the `Import-Csv` cmdlet and parameters.

- The `Import-Csv` cmdlet is going to load the given file into memory to be usable by PowerShell. It will assign each line and field as an input to be read.
  - All of the contents of this file will then become available to our script.
3. Now we need a `foreach` loop.
- Start typing a PowerShell `foreach` loop template:

Unset

```
$csv = Import-Csv -Path .\chocodemo.csv
foreach () {

}
```

- 4.
- Set the condition within the parentheses. In this case, we're calling the CSV file with a variable and reading each line with another.
- Enter `$line in $csv` in the parentheses `()`.
  - This specifies that for every line in the CSV file, `foreach` will execute the following code block with each line's contents.
5. Enter the code that will be executed on each line.
- Enter `Write-Output $line` in the curly brackets `{}`:

Unset

```
$csv = Import-Csv -Path .\chocodemo.csv
foreach ($line in $csv) {
 Write-Output $line
}
```

- Explain the syntax:

- Our `foreach` loop will grab the contents of each line in our CSV file and assign it to the `$line` variable.
- Then, for every line that gets assigned to the `$line` variable, the `Write-Output` cmdlet will print the contents.
- This process will repeat, or loop, until there are no more lines left to be assigned to the `$line` variable.

6. Run the script to verify that it prints out each line of our CSV file in the terminal.

- Type Ctrl+O and press Enter to save the file in Nano.
- Switch to the PowerShell window.
- Run the script using the command `.\removebloat.ps1`:

Unset

```
PS C:\Users\azadmin\Documents\Demo\choco> .\removebloat.ps1
```

```
name description
---- -
itunes Apple's music application
vlc An application for encoding and playing videos
```

- For every line in this CSV file, PowerShell is assigning that line to the variable `$line` and printing it out. After a line is printed out, the loop will continue until all lines have been printed by the `Write-Output` cmdlet.

Our `chocodemo.csv` file has multiple rows containing multiple attributes, such as `name` and `description`. Sometimes we only want to loop through the Windows Store application `name` or `description`.

This is made possible by appending `.name` to the `$line` variable. Your script will return the matching field or attribute entries instead of entire lines. In this case, it returns the application name.

1. Return to the Git Bash window and use Nano to open the script.

- Add `.name` after the `Write-Output $line` to show the following:

Unset

```
$csv = Import-Csv -Path .\chocodemo.csv
foreach ($line in $csv) {
 Write-Output $line.name
}
```

- Enter Ctrl+O to save your script file.
- Return to the PowerShell window and run the script again.
- Your output should be similar to:

Unset

```
PS C:\Users\azadmin\Documents\Demo\choco> .\removebloat.ps1
itunes
vlc
```

2.

We can retrieve descriptions instead of names.

- Using Git Bash, change the `.name` to `.description`, save, and re-run the script in the PowerShell window.

Unset

```
$csv = Import-Csv -Path .\chocodemo.csv
foreach ($line in $csv) {
 Write-Output $line.description
}
```

- Verify the output:

Unset

```
PS C:\Users\azadmin\Documents\Demo\choco> .\removebloat.ps1
Apple's music application
An application for encoding and playing videos
```

3.

Finally, let's combine the concepts of importing CSV files and creating `foreach` loops with our `choco uninstall` command.

- Return to your script in Git Bash and change the code block from:

- `Write-Output $line.description`

to:

- `choco uninstall -y $line.name.`

- Save with Ctrl+O but do not run the script yet.

Unset

```
$csv = Import-Csv -Path .\chocodemo.csv
foreach ($line in $csv) {
 choco uninstall -y $line.name
}
```

- When we execute this script, we should expect to see the output of the Choco package manager uninstalling the application packages in the CSV file.
- Run the script in PowerShell to verify the output.
- **Note:** There are multiple uninstalls with `itunes` due to the nature of the extra applications it comes with, like its mobile device integration.

We have now written our first PowerShell script that:

- Uses conditionals with `foreach` loops.
- Imports files.
- Uses the Choco package manager to remove unwanted system packages.

For the activity that follows, you'll apply these newly learned skills to remove the rest of the unwanted Choco packages from the Windows RDP host machine.

---

## 10. Activity: Removing Unnecessary Packages with PowerShell

-  M7.2 Activity File: Removing Unnecessary Packages with PowerShell
- [Choco CSV File](#)
- [Syntax Reference and Helpful Hints](#)

**:warning: Shut Down Your Machine :warning:**

Remember to shut down your Hyper-V virtual machine and Windows RDP host machine. You will need the remaining hours for the next lesson and Challenge.

---

## 11. Activity Review: Removing Unnecessary Packages with PowerShell

---

© 2024 edX Boot Camps LLC. Confidential and Proprietary. All Rights Reserved.